

BÀI TẬP VỀ NHÀ – Ôn tập Bài 13 → 16

PHẦN A – Lý thuyết

Trả lời ngắn gọn bằng lời của bạn.

1. Vì sao với signal ta phải dùng dạng tách rời [ngModel] + (ngModelChange) mà không dùng được cú pháp gộp [(ngModel)]?
 - Vì [(ngModel)] yêu cầu biến phải được gán giá trị trực tiếp (bằng dấu =). Trong khi đó, Signal trong Angular 17+ không phải là biến thường mà là một hàm, muốn thay đổi nó ta bắt buộc phải gọi phương thức .set() hoặc .update()
2. \$event trong (ngModelChange)="keyword.set(\$event)" là gì?
 - \$event chính là giá trị mới (kiểu chuỗi - string) mà người dùng vừa gõ vào ô input, được phát ra (emit) từ sự kiện ngModelChange
3. Phân biệt map(), filter() và .find(): mỗi cái trả về gì?
 - .map(): Trả về một mảng mới sau khi đã biến đổi từng phần tử.
 - .filter(): Trả về một mảng mới chứa các phần tử thỏa mãn điều kiện lọc.
 - .find(): Trả về một phần tử đầu tiên thỏa mãn điều kiện (hoặc undefined nếu không tìm thấy).
4. Vì sao khi tìm kiếm nên gọi toLowerCase() ở cả hai bên so sánh?
 - Để đưa cả từ khóa tìm kiếm lẫn chuỗi cần tìm về cùng dạng chữ thường, giúp việc tìm kiếm không bị phân biệt chữ hoa / chữ thường.
5. providedIn: 'root' nghĩa là gì? Nếu không có nó thì chuyện gì xảy ra khi hai component cùng dùng một service?
 - providedIn: 'root' giúp đăng ký Service này thành một "Singleton" (chỉ có duy nhất 1 bản sao trong toàn bộ ứng dụng). Nếu bỏ đi, khi tiêm vào nhiều component khác nhau, mỗi component sẽ tự tạo ra một bản sao Service độc lập, dẫn đến việc dữ liệu bị phân mảnh và không đồng bộ với nhau.
6. asReadonly() dùng để làm gì? Vì sao service nên giữ signal gốc private và chỉ cho ra ngoài bản chỉ đọc?
 - asReadonly() tạo ra một bản chỉ đọc (không thể dùng .set hay .update) của signal. Phải giữ signal gốc private để đảm bảo **tính đóng gói**: chỉ có Service mới có quyền thay đổi dữ liệu, ngăn không cho các Component bên ngoài vô tình sửa dữ liệu gốc.
7. Kể ra ba nơi khác nhau một thứ có thể "đi vào" component: inject(), mảng imports, và import thường. Cho một ví dụ cho mỗi nơi.
 - inject(): Tiêm Service / dependency hệ thống (VD: DrinkService, Router). Mảng imports: Nhập các module/component để dùng trong HTML (VD: FormsModule, RouterLink). import thường (trên cùng file): Khai báo kiểu dữ liệu/hàm TS (VD: DrinkModel, signal, computed)
8. inject() gọi được ở đâu và không gọi được ở đâu? Lỗi gì xảy ra nếu gọi sai chỗ?
 - Chỉ được gọi ở khu vực khai báo thuộc tính class (property initialization) hoặc trong constructor. Nếu gọi bên trong một hàm (ví dụ như ngOnInit hoặc hàm xử lý click), Angular sẽ báo lỗi NG0203
9. Vì sao pathMatch: 'full' là bắt buộc với route rỗng path: ""?
 - Vì chuỗi rỗng "" luôn là phần tiền tố của mọi URL. Nếu không có pathMatch: 'full' (yêu cầu khớp chính xác 100%), Router sẽ hiểu lầm mọi đường dẫn đều khớp với chuỗi rỗng, gây ra lỗi vòng lặp hoặc chuyển hướng sai.

10. Phân biệt `href="/drinks/2"` và `[routerLink]="['/drinks', 2]"`. Hậu quả của việc dùng sai?
- `href` khiến trình duyệt **tải lại toàn bộ trang web**, làm mất sạch state và dữ liệu đang lưu trong bộ nhớ. `[routerLink]` dùng Router của Angular để hoán đổi Component ngay bên dưới mà không cần tải lại trang (kiến trúc SPA)
11. `route.params` và `route.paramMap` khác nhau thế nào? Cách lấy giá trị của mỗi loại?
- `params`: Là một object JavaScript thuần, lấy giá trị bằng ngoặc vuông: `params()['id']`.
 - `paramMap`: Là đối tượng nâng cao có cung cấp sẵn các phương thức, lấy giá trị bằng hàm: `paramMap().get('id')`. (Nên dùng `paramMap`)
12. Vì sao tham số lấy từ URL luôn phải đưa qua `Number()` trước khi so sánh với `id`?
- Bởi vì mọi thứ nằm trên thanh URL đều được hiểu là **chuỗi** (String). Nếu ID trong model là Số (Number), so sánh `===` giữa `"1"` và `1` sẽ ra kết quả `false`, dẫn tới lỗi không tìm thấy món
13. So sánh Template-driven form (`ngModel`) và Reactive form (`formGroup`): mỗi loại phù hợp với tình huống nào?
- **Template-driven (`ngModel`)**: Phù hợp cho form đơn giản (chỉ có 1 ô tìm kiếm, form đăng nhập 2 ô).
 - **Reactive form (`FormGroup`)**: Phù hợp cho form phức tạp, nhiều điều kiện validate ràng buộc, form động (như trang Add Drink).
14. Vì sao dùng `(ngSubmit)` trên thẻ `<form>` tốt hơn `(click)` trên nút gửi?
- Vì `(ngSubmit)` lắng nghe được cả thao tác nhấn phím **Enter** trên bàn phím lúc đang gõ ô input. Dùng `(click)` thì người dùng bắt buộc phải dùng chuột bấm vào nút mới gửi được
15. `value` và `getRawValue()` khác nhau ở kiểu trả về thế nào? Và `fb.nonNullable.group()` giải quyết thêm vấn đề gì?
- `.value` sẽ bỏ qua các ô bị disabled, còn `.getRawValue()` thì lấy sạch toàn bộ các ô.
 - `fb.nonNullable` giúp loại bỏ kiểu null mặc định của FormBuilder, giúp dữ liệu lấy ra luôn an toàn ở định dạng string / number
16. Vì sao trong service không được dùng `.push()` để thêm phần tử vào mảng của signal?
- Vì `.push()` chỉnh sửa trực tiếp trên ô nhớ cũ. Signal phát hiện ô nhớ không đổi nên sẽ không báo cho giao diện cập nhật. Phải dùng `.update()` tạo mảng mới hoàn toàn: `current => [...current, newItem]`

PHẦN B – Tìm lỗi trong code

Mỗi đoạn có đúng một lỗi. Chỉ ra lỗi, giải thích tại sao sai, và viết lại cho đúng.

Lỗi số 1

```
<input [(ngModel)]="keyword" type="search" />
```

Đoạn code sai: `<input [(ngModel)]="keyword" type="search" />`

Tại sao sai: Cú pháp "chuỗi trong hộp" [(ngModel)] là tàn dư của Angular thế hệ cũ. Nó hoạt động bằng cách gán thẳng giá trị (bằng dấu =) vào biến. Tuy nhiên, keyword của ta đang là một **Signal**. Signal không cho phép gán trực tiếp kiểu keyword = "trà sữa", mà bắt buộc phải thông qua hàm .set().

Cách sửa: Tách cú pháp gộp thành 2 chiều riêng biệt (đọc bằng [] và ghi bằng ()):

HTML

```
<input [ngModel]="keyword()" (ngModelChange)="keyword.set($event)" type="search" />
```

Lỗi số 2

```
import { Component, signal } from '@angular/core';

@Component({
  selector: 'app-drink-list',
  imports: [],
  templateUrl: './drink-list.html',
  styleUrls: ['./drink-list.css'],
})
export class DrinkList {
  protected readonly keyword = signal<string>('');
}

<!-- HTML -->
<input [ngModel]="keyword()" (ngModelChange)="keyword.set($event)" />
```

Đoạn code sai: Trong @Component để imports: [] nhưng dưới HTML lại xài [ngModel].

Tại sao sai: ngModel không phải là HTML thuần, nó là một directive thuộc về FormsModule của Angular. Nếu bạn dùng Standalone Component mà quên khai báo "tôi cần dùng đồ của FormsModule" trong mảng imports, Angular sẽ báo lỗi không hiểu ngModel là cái gì.

Cách sửa:

TypeScript

```
import { FormsModule } from '@angular/forms';

@Component({
  imports: [FormsModule], // Phải khai báo ở đây
  // ...
})
```

Lỗi số 3

```
protected readonly filteredDrinks = computed(() => {
  const key = this.keyword().toLowerCase().trim();
  return this.drinks().filter((drink) => drink.name.includes(key));
});
```

Đoạn code sai: drink.name.includes(key)

Tại sao sai: Bạn đã cẩn thận gọi .toLowerCase() để hạ từ khóa key xuống chữ thường, nhưng bạn lại quên hạ chữ của drink.name.

- Ví dụ: key là "matcha". Tên món là "Trà sữa Matcha". Lệnh "Trà sữa Matcha".includes("matcha") sẽ trả về false do chữ M viết hoa không khớp với chữ m viết thường.

🔗 **Cách sửa:** Ép cả 2 về về chữ thường trước khi so sánh:

TypeScript

```
return this.drinks().filter((drink) => drink.name.toLowerCase().includes(key));
```

Lỗi số 4

```
import { Injectable, signal } from '@angular/core';
import { MOCK_DRINKS } from './mock-drinks';

@Injectable()
export class DrinkService {
  private readonly drinksState = signal(MOCK_DRINKS);
  readonly drinks = this.drinksState.asReadonly();
}
```

Đoạn code sai: @Injectable()

Tại sao sai: Khai báo thể này Service vẫn hoạt động, nhưng nó **không phải là Singleton**. Mỗi lần một Component gọi inject(DrinkService), Angular lại đẻ ra một bản sao Service mới toanh. Dẫn đến việc dữ liệu trà sữa bị phân mảnh, Component A thêm món mới nhưng Component B không nhìn thấy.

Cách sửa: Khai báo nó tồn tại duy nhất ở cấp cao nhất (root):

TypeScript

```
@Injectable({ providedIn: 'root' })
```

Lỗi số 5

```
export class DrinkList {
  protected readonly selectedDrink = signal<DrinkModel>(
    this.drinkService.drinks()[0]
  );
  private readonly drinkService = inject(DrinkService);
}
```

Đoạn code sai:

TypeScript

```
protected readonly selectedDrink = signal(this.drinkService.drinks()[0]);
private readonly drinkService = inject(DrinkService);
```

Tại sao sai: Trong một class của TypeScript, code được chạy từ trên xuống dưới. Bạn đang cố dùng this.drinkService để lấy dữ liệu cho selectedDrink, trong khi phải tới dòng bên dưới thẳng drinkService mới được sinh ra (inject). Trình duyệt sẽ quăng lỗi undefined.

Cách sửa: Đảo dòng inject lên trên cùng:

TypeScript

```
private readonly drinkService = inject(DrinkService);
protected readonly selectedDrink = signal(this.drinkService.drinks()[0]);
```

Lỗi số 6

```
export class DrinkDetail {
  protected goBack(): void {
    const router = inject(Router);
    router.navigate(['/drinks']);
  }
}
```

Đoạn code sai: Đặt `const router = inject(Router);` vào bên trong một hàm `goBack()`.

Tại sao sai: Cơ chế `inject()` của Angular cực kỳ nghiêm ngặt. Nó **chỉ được phép chạy khi Component đang được khởi tạo** (tức là nằm ở chỗ khai báo thuộc tính class, hoặc trong constructor). Nếu bạn nhét nó vào một hàm (chỉ chạy khi người dùng bấm nút click), Angular đã đóng cổng tiêm rồi, nó sẽ báo lỗi NG0203 ngay lập tức.

Cách sửa: Đem ra ngoài khai báo làm thuộc tính của class:

TypeScript

```
export class DrinkDetail {
  private readonly router = inject(Router); // Khai báo ở đây

  protected goBack(): void {
    this.router.navigate(['/drinks']); // Dùng ở đây
  }
}
```

Lỗi số 7

```
export const routes: Routes = [
  { path: 'drinks', component: DrinkList },
  { path: 'drinks/:id', component: DrinkDetail },
  { path: 'drinks/new', component: AddDrink },
  { path: '', redirectTo: 'drinks', pathMatch: 'full' },
];
```

Đoạn code sai: Đặt `path: 'drinks/:id'` lên TRƯỚC `path: 'drinks/new'`.

Tại sao sai: Router của Angular đọc danh sách từ trên xuống dưới, thấy ai khớp là nó hốt luôn. Tham số `:id` là một "biến động" (đại diện cho mọi chuỗi). Khi bạn vào URL `drinks/new`, Router thấy từ `"new"` nhét vừa vào chỗ `":id"`, thế là nó tải luôn trang `DrinkDetail` (Chi tiết món) và gán `id = "new"`, làm ứng dụng bị lỗi. Trang `AddDrink` mãi mãi không được gọi.

Cách sửa: Các route **Cụ thể** phải đặt trước, route **Động (có dấu hai chấm)** phải đặt sau:

TypeScript

```
{ path: 'drinks/new', component: AddDrink }, // Đặt lên trước
{ path: 'drinks/:id', component: DrinkDetail },
```

Lỗi số 8

```
private readonly params = toSignal(this.route.params);
protected readonly selectedDrink = computed(() => {
  const id = Number(this.params().get('id'));
  return this.drinkService.getDrinkById(id);
});
```

Đoạn code sai: `const id = Number(this.params().get('id'));` (trong khi đang bắt `this.route.params`).

Tại sao sai: Thuộc tính `.params` trả về một Object thuần của Javascript (vd: `{ id: '1' }`). Một Object thuần thì không hề có phương thức `.get()`. Phương thức `.get()` chỉ tồn tại bên trong `.paramMap` (một cấu trúc dữ liệu xịn hơn kiểu Map).

Cách sửa:

TypeScript

// Sửa chỗ này: `paramMap` thay vì `params`

```
private readonly params = toSignal(this.route.paramMap);
```

Lỗi số 9

```
protected readonly selectedDrink = computed(() => {
  const id = this.params()?.get('id');
  return this.drinkService.getDrinkById(id);
});
```

Đoạn code sai: `const id = this.params()?.get('id');` sau đó ném thẳng vào `getDrinkById(id)`.

Tại sao sai: Bất cứ thứ gì nằm trên thanh URL (ví dụ `drinks/2`) đều được trình duyệt hiểu là **Chữ (String)**. Lệnh `.get('id')` sẽ trả về chữ `"2"`. Trong khi database của bạn dùng **Số (Number)** (`id: 2`). Nếu mang chữ `"2"` đi tìm số 2, Javascript sẽ bảo là "không bằng nhau", dẫn tới không tìm thấy món.

Cách sửa: Luôn bọc `Number()` để ép về số:

TypeScript

```
const id = Number(this.params()?.get('id'));
```

Lỗi số 10

```
addDrink(newDrink: DrinkModel): void {
  this.drinksState().push(newDrink);
}
```

Đoạn code sai: `this.drinksState().push(newDrink);`

Tại sao sai: `.push()` là hàm sửa trực tiếp mảng hiện tại. Đối với Signal, nếu bạn chọc thẳng vào mảng gốc và nhét thêm đồ vào, Signal sẽ không nhận ra sự thay đổi (vì vùng nhớ của mảng không đổi). Giao diện sẽ KHÔNG tự động vẽ lại danh sách mới.

Cách sửa: Bắt buộc dùng `.update()` và tạo ra mảng mới (dùng cú pháp spread `...` của ES6):

TypeScript

```
this.drinksState.update(current => [...current, newDrink]);
```

Lỗi số 11

```
@if (selectedDrink(); as drink) {  
  <h2>{{ selectedDrink().name }}</h2>  
}
```

Đoạn code sai:

HTML

```
@if (selectedDrink(); as drink) {  
  <h2>{{ selectedDrink().name }}</h2> <!-- Vẫn gọi lại hàm signal -->  
}
```

Tại sao sai: Hàm `selectedDrink()` có thể trả về `DrinkModel` hoặc `undefined`. Lệnh `@if` đã kiểm tra an toàn và gán kết quả vào biến `drink`. Nhưng bên trong thẻ `<h2>`, bạn lại gọi `selectedDrink()` một lần nữa. TypeScript sẽ báo lỗi vì nó không thể chắc chắn lần gọi thứ 2 này kết quả có bị `undefined` hay không.

Cách sửa: Đã gán vào biến `as drink` rồi thì bên trong dùng đúng biến đó:

HTML

```
<h2>{{ drink.name }}</h2>
```

Lỗi số 12

```
protected readonly drinkForm = this.fb.group({  
  name: ['', Validators.required],  
});  
private readonly fb = inject(FormBuilder);
```

Đoạn code sai: Khai báo biến `drinkForm` dùng `this.fb`, trong khi lệnh `inject(FormBuilder)` nằm ở dòng bên dưới.

Tại sao sai: Lỗi y chang lỗi số 5. Class khởi tạo từ trên xuống, `fb` chưa sinh ra thì không thể cấu hình Form được.

Cách sửa: Đảo vị trí lệnh `inject` lên trên cùng:

TypeScript

```
private readonly fb = inject(FormBuilder);  
protected readonly drinkForm = this.fb.group({ ... });
```